

MANUAL TÉCNICO

Sistema de Gestão de Mercado

Guri Atacadista

Arquitetura, instalação, operação, banco de dados e manutenção

Versão	1.0
Data de emissão	02 de novembro de 2026

Material de apoio para desenvolvimento, implantação e suporte técnico.

Apresentação

Este manual descreve a estrutura técnica do sistema Guri Atacadista, um aplicativo desktop de gestão de mercado desenvolvido em C# com Windows Forms e banco de dados Microsoft SQL Server. O documento foi elaborado a partir da leitura do código-fonte, dos arquivos de configuração, do script SQL e do README incluídos no repositório enviado.

> **Escopo da análise**
>
> A documentação representa o que está presente nos arquivos analisados. Não foi possível executar a aplicação nem validar uma instância real do SQL Server neste ambiente. Por isso, os fluxos descritos como previstos são diferenciados dos pontos que exigem correção ou homologação no código atual.

Como utilizar este manual

Desenvolvedores devem priorizar as seções de arquitetura, configuração, banco de dados e observações técnicas. Administradores de banco devem consultar o modelo relacional, os gatilhos e as procedures. Pessoas responsáveis pela operação podem utilizar os fluxos funcionais, respeitando os alertas sobre funcionalidades ainda incompletas.

Sumário

- 1. Visão geral e requisitos técnicos
- 2. Arquitetura e organização do código
- 3. Módulos e fluxos funcionais
- 4. Modelo de dados
- 5. Regras de negócio no SQL Server
- 6. Instalação e configuração
- 7. Compilação e execução
- 8. Operação do sistema
- 9. Pontos de atenção identificados no código
- 10. Manutenção, testes e solução de problemas
- 11. Referência rápida de arquivos e objetos SQL

1. Visão geral e requisitos técnicos

O sistema tem como objetivo apoiar rotinas de caixa, cadastro e consulta de clientes, gestão de funcionários, fornecedores, produtos, lotes, compras e relatórios de estoque e movimentação financeira. A aplicação possui interface gráfica Windows Forms e persiste os dados em tabelas relacionais com prefixo GA_.

Componente	Tecnologia / configuração identificada
Linguagem	C#
Interface	Windows Forms (aplicativo desktop)
Framework	.NET Framework 4.7.2
IDE / solução	Visual Studio; solução Guri_atacarejo.sln
Persistência	Microsoft SQL Server; configuração de referência para SQL Server LocalDB
Acesso a dados	DataSets tipados, TableAdapters e System.Data.SqlClient
Formato de transações	Stored procedures, triggers e transações T-SQL
Dados de entrada em lote	JSON textual processado por OPENJSON nas procedures

Requisitos para desenvolvimento e execução

- Sistema operacional Windows compatível com Windows Forms e .NET Framework 4.7.2.

- Visual Studio com a carga de trabalho de desenvolvimento desktop .NET e o Developer Pack do .NET Framework 4.7.2.
- SQL Server Express LocalDB instalado para o cenário local documentado no README; SQL Server Management Studio é recomendado para administrar e executar o script.
- Para uso de OPENJSON, utilizar SQL Server 2016 ou superior e compatibilidade do banco igual ou superior ao nível 130.

> **Banco vazio após a instalação**
>
> O script SuperMercado.sql cria o catálogo e as estruturas, mas não contém INSERTs de dados iniciais para cargos, departamentos, categorias, fornecedores, produtos ou funcionários. A carga inicial dessas tabelas é uma etapa administrativa obrigatória.

2. Arquitetura e organização do código

A solução adota uma arquitetura desktop concentrada em formulários: os formulários apresentam os dados, tratam eventos e executam parte das regras de negócio. A persistência é feita diretamente por TableAdapters/DataSets ou por comandos SqlClient. O banco completa a lógica com constraints, gatilhos e procedures.

Camada lógica	Responsabilidade	Exemplos no repositório
Apresentação	Telas, menus, formulários modais, grids, cards e validação de campos.	forms/frmPainel.cs; forms/frmCompra.cs; demais forms/
Sessão	Mantém os identificadores do funcionário e os dados temporários do cliente logado.	Sessao/Sessao.cs; Sessao/SessaoClientes.cs
Acesso a dados	Consulta e grava dados por consultas parametrizadas, DataSets e TableAdapters.	DataSet.xsd; DataSet.Designer.cs; SqlClient nos forms
Banco de dados	Restrições, totais calculados, movimentação de estoque e transações.	SuperMercado.sql

Fluxo simplificado de comunicação

Etapa	Descrição
1. Interface	O usuário aciona um formulário, botão ou item de menu.
2. Código C#	O evento valida os campos e prepara os parâmetros da operação.
3. Acesso a dados	Um TableAdapter ou SqlCommand envia a consulta ou procedure ao SQL Server.
4. Banco	Constraints, triggers e procedures validam e persistem os registros.
5. Retorno	O formulário apresenta a confirmação, atualiza os grids ou informa o erro.

Estrutura de pastas e arquivos

Caminho	Conteúdo
Guri_atacarejo.sln	Solução principal do Visual Studio.
Guri_atacarejo/Guri_atacarejo.csproj	Projeto WinForms e referências do framework.
Guri_atacarejo/forms/	Formulários de login, caixa, cadastros, compras e relatórios.

Caminho	Conteúdo
Guri_atacarejo/Sessao/	Classes estáticas de sessão.
Guri_atacarejo/DataSet.xsd e arquivos DataSet*	Esquemas e código gerado dos DataSets/TableAdapters.
Guri_atacarejo/Properties/	Configurações, recursos e metadados do aplicativo.
Guri_atacarejo/Resources/ e Assets/	Logotipos e imagens utilizadas pela interface.
SuperMercado.sql	Criação do catálogo, tabelas, triggers, procedures e view.
README.md	Descrição funcional e instruções de configuração local.

Padrões de implementação

- As classes de formulário seguem a convenção frmNome; controles reutilizáveis são implementados como UserControls, por exemplo CardProduto e CardCarrinho.
- ResponsividadeForm aplica redimensionamento proporcional de controles em formulários que a utilizam.
- RoundTextBox, RoundComboBox, RoundDate e RoundControl implementam componentes visuais personalizados.
- O acesso a dados não está concentrado em um repositório único. Há consultas SQL e cadeias de conexão distribuídas por formulários e configurações geradas.

3. Módulos e fluxos funcionais

3.1 Inicialização e autenticação do funcionário

O ponto de entrada está em Program.cs. Ele habilita estilos visuais do Windows Forms e abre SplashScreen. A splash screen usa um Timer de 4.000 ms e, ao terminar, abre FrmLogon. O login consulta GA_Funcionarios por Nome e Senha e registra os dados retornados na classe estática Sessao.

Dado de sessão	Uso
Sessao.IdFuncionario	Identifica o funcionário em operações que registram compras e vendas.
Sessao.Usuario	Guarda o nome do funcionário autenticado.
Sessao.NivelFuncionario	É consultado para exibir ou ocultar menus de gestão e relatórios.

3.2 Caixa / venda

O formulário principal frmPainel apresenta o carrinho em um DataGridView, os detalhes do produto selecionado, o cliente, a forma de pagamento, o total e as ações de adicionar, alterar, cancelar e finalizar. Para adicionar um item, frmAddProdVenda recebe código de produto, código de lote e quantidade. A consulta GetProdutoOuLote recupera os dados do produto e a data de validade; o painel consulta GA_Estoque e rejeita quantidades superiores ao saldo do lote.

Ao finalizar, o código monta um JSON com FKLote e Quantidade e chama a procedure usp_RegistrarVenda. A procedure cria a venda e insere os itens. Os gatilhos calculam o total e baixam o estoque. O sistema exibe a mensagem devolvida pelo banco e limpa o carrinho quando a confirmação é bem-sucedida.

> **Atenção à finalização atual**
>
> No código analisado, a chamada da procedure utiliza FKCliente = 1, FKFuncionario = 1 e FormaPagamento = "Pix" fixos. Portanto, o vínculo com o funcionário autenticado, o cliente selecionado e a opção escolhida na tela ainda não está integrado corretamente. Consulte a seção 9 antes de utilizar o caixa em produção.

3.3 Clientes

O cadastro de clientes está em frmRegistro_Cliente.cs e grava Nome, CPF, Genero, Email e Celular em GA_Clientes. O código formata CPF e telefone e verifica campos obrigatórios e padrões básicos de e-mail. A classe SessaoClientes mantém temporariamente nome e CPF do cliente identificado.

Há também FrmLoginCliente, que consulta o CPF por meio de um TableAdapter. Entretanto, o TableAdapter desse fluxo está associado ao catálogo SuperMercado, e a tela chama Application.Run com outro formulário em vez de apenas preencher a sessão existente. A integração precisa ser revisada.

3.4 Funcionários e controle de acesso

frmCadastroFuncionario.cs carrega cargos e departamentos, valida campos cadastrais e grava o funcionário em GA_Funcionarios. A tabela GA_Cargos contém um campo Nivel, utilizado para definir permissões. O objetivo descrito no README é permitir que o nível 1 utilize o caixa e que níveis superiores também acessem cadastros e relatórios.

A leitura de permissão no login, contudo, consulta FKCargo e atribui esse identificador a Sessao.NivelFuncionario. Como FKCargo é a chave do cargo, e Nivel é um atributo diferente, o controle pode divergir da regra pretendida quando o identificador do cargo não coincidir com seu nível.

3.5 Produtos, fornecedores e lotes

- Fornecedores: frmRegistroFornecedor valida nome, CNPJ, e-mail e endereço e utiliza GA_FornecedorTableAdapter.
- Produtos: frmCadastroProduto é aberto pelo menu de gestão, mas o comando de gravação atual insere na tabela Produtos, que não faz parte do esquema GA_ do script. A tela precisa ser alinhada à tabela GA_Produtos e às chaves de fornecedor e categoria.
- Lotes: frmRegistroLote recebe fornecedor, produto, quantidade e validade. O código atual aponta para o catálogo SuperMercado e insere somente em GA_Lote, sem criar o registro correspondente em GA_Estoque.

3.6 Compras e relatórios

frmCompra.cs monta itens de compra por produto e fornecedor, utiliza a sessão do funcionário e chama usp_RegistrarCompra. A procedure cria uma compra, gera um lote por item, cria os registros de estoque e grava os itens da compra dentro de uma transação.

O relatório de compras está em frmRelatorioCompra.cs e utiliza TableAdapters para listar compras por intervalo de datas e detalhar seus itens. O relatório de estoque está em frmRelatorioEstoque.cs: soma quantidades dos lotes por produto e, após a seleção de um produto, consulta operações de compra e venda e permite filtrar a visualização por tipo e período.

> **Tela prototipada**
>
> O formulário frmAdicionar.cs contém uma lista fixa de produtos de demonstração e os botões de confirmar/cancelar apenas navegam para o painel. Não há gravação de dados nessa confirmação; não deve ser confundido com o cadastro persistente de GA_Produtos.

4. Modelo de dados

O script cria o catálogo db_250405 e 13 tabelas principais com prefixo GA_. As chaves primárias são, em geral, colunas inteiras Identity. As relações são protegidas por foreign keys e as principais quantidades e valores possuem constraints de não negatividade.

Tabela	Finalidade e campos relevantes
GA_Clientes	Cadastro de cliente; CPF único; nível de fidelidade e total acumulado.
GA_Cargos	Nome do cargo, salário-base e nível de acesso.
GA_Departamentos	Cadastro de departamentos.
GA_Funcionarios	Dados pessoais, cargo, departamento, senha, salário e status funcional.
GA_Fornecedor	Cadastro de fornecedor; CNPJ único, contatos e endereço.

Tabela	Finalidade e campos relevantes
GA_Categoria	Categorias de produtos.
GA_Produtos	Catálogo de produtos; FKFornecedor, FKCategoria, nome, unidade de medida e preço.
GA_Lote	Lote de um produto, fornecedor, quantidade, validade e preço total do lote.
GA_Estoque	Uma linha por lote; quantidade atual, quantidade mínima e status.
GA_Compra	Cabeçalho da compra: fornecedor, funcionário, data, valor, pagamento e status.
GA_ItensCompra	Itens da compra: lote, quantidade, preço unitário e subtotal.
GA_Venda	Cabeçalho da venda: cliente opcional, funcionário, data, total, pagamento e status.
GA_ItensVenda	Itens vendidos: lote, quantidade, preço unitário, subtotal e indicador Cancelado.

Relacionamentos principais

- GA_Funcionarios.FKCargo -> GA_Cargos.IdCargo e GA_Funcionarios.FKDepartamento -> GA_Departamentos.IdDepartamento.
- GA_Produtos.FKFornecedor -> GA_Fornecedor.IdFornecedor e GA_Produtos.FKCategoria -> GA_Categoria.IdCategoria.
- GA_Lote referencia fornecedor e produto; GA_Estoque referencia um lote por meio de FKLote, que é UNIQUE.
- GA_Compra referencia fornecedor e funcionário; GA_ItensCompra referencia compra e lote.
- GA_Venda referencia funcionário e, opcionalmente, cliente; GA_ItensVenda referencia venda e lote.

Regras e restrições de integridade

- CPF de cliente e funcionário e CNPJ de fornecedor são definidos como únicos.
- Gênero aceita M, F ou O nas tabelas de cliente e funcionário.
- Preços, quantidades e quantidades mínimas possuem verificações para impedir valores negativos nas tabelas em que a constraint está definida.
- Status do funcionário aceita Ativo, Inativo ou Afastado; status de compra e venda aceitam Pendente, Pago ou Cancelado.
- Cada registro de GA_Estoque deve corresponder a um lote existente, e o banco permite no máximo um registro de estoque por lote.

Representação lógica resumida

Entidade principal	Entidades dependentes
Cliente	Venda
Cargo / Departamento	Funcionário
Fornecedor / Categoria	Produto
Produto / Fornecedor	Lote
Lote	Estoque, ItensCompra e ItensVenda
Compra	ItensCompra
Venda	ItensVenda

5. Regras de negócio no SQL Server

5.1 Gatilhos (triggers)

Trigger	Evento e efeito
trg_UpdateTotalVenda	Após INSERT/UPDATE/DELETE em GA_ItensVenda, recalcula ValorTotal das vendas afetadas.
trg_UpdateTotalCompra	Após INSERT/UPDATE/DELETE em GA_ItensCompra, recalcula ValorTotal das compras afetadas.
trg_BaixarEstoqueVenda	Após inserir itens de venda, verifica a existência do estoque e saldo suficiente; depois reduz a quantidade por lote.
trg_AdicionarEstoqueCompra	Após inserir itens de compra, exige registro de estoque e adiciona a quantidade comprada.
trg_AtualizarEstoqueMinimo	Após inserir/atualizar GA_Estoque, define o status como Indisponível, Próximo ao mínimo ou Disponível.
trg_DevolverEstoque_ItemCancelado	Quando Cancelado muda para 1, devolve ao estoque a quantidade dos itens afetados.
trg_CancelarItensAoCancelarVenda	Quando uma venda passa para Cancelado, marca os itens ativos como cancelados.
trg_AtualizarTotalComprasCliente	Recalcula o total de vendas não canceladas associado ao cliente.
trg_AtualizarPrecoLote	Após inserir/atualizar itens de compra, soma os subtotais e atualiza o preço total do lote.

5.2 Procedures (procedimentos armazenados)

Procedure	Parâmetros / comportamento
usp_RegistrarVenda	Recebe cliente opcional, funcionário, forma de pagamento e JSON com FKLote e Quantidade. Valida estoque, cria venda e itens em transação e retorna o IdVenda e mensagem.
usp_RegistrarCompra	Recebe fornecedor, funcionário, forma de pagamento, data de entrega e JSON com FKProduto, Quantidade, DataValidade e QuantidadeMinima. Cria compra, lotes, estoque e itens.
usp_CancelarVenda	Recebe IdVenda; marca a venda como cancelada. Os gatilhos cancelam itens e devolvem estoque.
usp_CancelarItensVenda	Recebe IdVenda e JSON com IDs de itens; cancela somente os itens selecionados e devolve suas quantidades.

Diferença entre README e script SQL

O README cita usp_RelatorioEstoqueCritico, mas essa procedure não aparece criada no SuperMercado.sql. O script disponibiliza a view vw_EstoqueCritico, que lista lotes com quantidade atual menor ou igual à quantidade mínima.

5.3 Formato dos dados JSON

As procedures utilizam OPENJSON para converter o texto JSON em linhas relacionais. Os nomes das propriedades precisam corresponder exatamente aos caminhos definidos no T-SQL.

Exemplo do conteúdo esperado por usp_RegistrarVenda:

```
[
  {"FKLote": 12, "Quantidade": 2},
  {"FKLote": 18, "Quantidade": 1}
]
```

Exemplo do conteúdo esperado por usp_RegistrarCompra:

```
[
  {"FKProduto": 4, "Quantidade": 50, "DataValidade": "2027-03-01", "QuantidadeMinima": 10}
]
```

Observação: a procedure de compra lê o preço de venda cadastrado em GA_Produtos.Preco para preencher PrecoUnitario. A propriedade Preco enviada pelo formulário não é consumida pela definição atual da procedure.

6. Instalação e configuração

6.1 Preparar o ambiente

1. Instale o .NET Framework 4.7.2 (runtime e, para compilar, Developer Pack).
2. Instale o SQL Server Express LocalDB. Para administração visual, instale também o SQL Server Management Studio.
3. Abra o SSMS e conecte-se à instância (localdb)\MSSQLLocalDB com autenticação do Windows.
4. Abra o arquivo SuperMercado.sql localizado na raiz do repositório e execute o script completo.
5. Verifique se o catálogo db_250405 foi criado e se as tabelas, triggers, procedures e a view estão disponíveis.

6.2 Verificar banco e compatibilidade

```
SELECT @@VERSION AS VersaoSQL;

SELECT name, compatibility_level
FROM sys.databases
WHERE name = 'db_250405';
```

Se o nível de compatibilidade for inferior a 130, avalie com o administrador atualizar o banco para um nível compatível com OPENJSON antes de utilizar as rotinas de venda e compra. Não aplique a alteração em ambiente compartilhado sem validar a versão do mecanismo SQL Server.

6.3 Carga inicial obrigatória

O script de instalação não cria dados de referência nem um usuário administrador. Antes do primeiro login, é necessário cadastrar ao menos um departamento, um cargo e um funcionário válido, além de popular as categorias e fornecedores necessárias ao catálogo de produtos. A carga deve respeitar as chaves estrangeiras e as regras de negócio da organização.

> **Atenção para a primeira autenticação**
>
> Não há credencial inicial documentada nem rotina de criação automática de administrador. O provisionamento inicial deve ser executado por um responsável pelo banco, em ambiente controlado. A senha é atualmente comparada diretamente com o valor armazenado, sem hash; isso precisa ser corrigido antes de uma implantação real.

6.4 Configuração da conexão

A configuração local de referência está em Guri_atacarejo/App.config e utiliza o nome Guri_atacarejo.Properties.Settings.ConnectionString:

```
Data Source=(localdb)\MSSQLLocalDB;
Initial Catalog=db_250405;
Integrated Security=True;
TrustServerCertificate=True;
```

A autenticação integrada utiliza a identidade Windows do usuário que executa o aplicativo. Esse usuário precisa ter permissões adequadas no catálogo db_250405. Para outro servidor, atualize a cadeia de conexão no App.config e revise também as configurações geradas e os TableAdapters que usam ConnectionString1.

6.5 Atenção às configurações divergentes

O repositório mantém referências antigas a outros servidores e ao catálogo SuperMercado. Há arquivos de configuração e TableAdapters que não apontam todos para db_250405. Há também uma cadeia com aparência de credencial SQL em Settings.settings; o valor não é reproduzido neste manual. Trate a credencial como exposta, rotacione-a caso ainda esteja ativa e remova segredos do código-fonte e do histórico do repositório.

7. Compilação e execução

7.1 Abrir e compilar

1. Extraia o repositório em uma pasta local com permissões de leitura e escrita.
2. Abra Guri_atacarejo.sln no Visual Studio.
3. Confirme que Guri_atacarejo é o projeto de inicialização.
4. Verifique a propriedade Target Framework: .NET Framework 4.7.2.
5. Confirme as referências do projeto e a cadeia de conexão usada no ambiente.
6. Selecione Debug ou Release e execute Rebuild Solution.
7. Inicie com F5 ou Ctrl+F5. O aplicativo abre a splash screen e depois a tela de login.

7.2 Saída compilada

O projeto configura o executável Guri_atacarejo.exe nas pastas bin\Debug ou bin\Release. A distribuição deve incluir o executável e os arquivos de configuração gerados pelo build, além de garantir o acesso ao SQL Server. Não distribua arquivos de desenvolvimento, arquivos PDB ou cópias de bancos de dados com informações reais sem avaliação de segurança.

7.3 Validação após a instalação

- Abrir o aplicativo e confirmar a transição da splash para o login.
- Validar conectividade com o banco e autenticação de funcionário de teste.
- Confirmar a presença dos cadastros básicos e os vínculos de cargo/departamento.
- Testar uma consulta de produto e estoque antes de tentar registrar vendas.
- Validar o ciclo completo de compra e venda em ambiente de teste, verificando totais e quantidades no banco.

8. Operação do sistema

8.1 Acesso do funcionário

1. Inicie o aplicativo; aguarde a splash screen encerrar.
2. Informe o nome e a senha registrados em GA_Funcionarios.
3. Após autenticação, o painel frmPainel é aberto. Os menus são exibidos conforme Sessao.NivelFuncionario.
4. Para encerrar a sessão, utilize Funcionários > Logout.

8.2 Registrar uma venda - fluxo previsto

1. No caixa, selecione Adicionar produto.
2. Informe o ID do produto, o ID do lote e a quantidade. Produto e lote precisam estar relacionados no banco.
3. O sistema consulta a validade, preço e unidade do produto e verifica o saldo de GA_Estoque para o lote.
4. Repita para os demais itens. Para alterar a quantidade, selecione a linha do carrinho e utilize Alterar produto. Quantidade zero remove o item.
5. Identifique o cliente, quando aplicável, e selecione a forma de pagamento.
6. Finalize a venda. A procedure valida os lotes e o banco atualiza o total e o estoque.

> **Operação condicionada à correção do código**
>
> A etapa 5 e a associação de funcionário/cliente ainda não são respeitadas pela chamada atual da procedure, que utiliza IDs fixos e Pix. O fluxo é documentado como comportamento esperado, não como garantia de funcionamento.

8.3 Registrar uma compra de mercadorias

1. Abra Mais opções > Lotes > Compra de lotes.
2. Selecione produtos, quantidades, fornecedores e datas de validade disponíveis na tela.
3. Confira os preços informados e o total apresentado.
4. Confirme a compra. O formulário agrupa os itens por fornecedor e chama usp_RegistrarCompra para cada grupo dentro de uma transação externa.
5. Após a confirmação, verifique GA_Compra, GA_ItensCompra, GA_Lote e GA_Estoque.

8.4 Cadastros e consultas

Menu / tela	Uso descrito
Clientes > Registrar Cliente	Cadastrar dados do cliente e CPF.
Clientes > Login Cliente	Localizar cliente pelo CPF; a integração atual requer revisão.
Mais opções > Registrar Funcionários	Cadastrar funcionário, cargo, departamento e senha.
Mais opções > Registrar produtos	A tela existe, mas o comando SQL atual não corresponde ao esquema GA_.
Mais opções > Registrar fornecedor	Cadastrar nome, CNPJ, e-mail, telefone e endereço.
Mais opções > Lotes > Registo de lotes	Criar lote; atualmente não cria o estoque associado.
Relatórios > Movimentação financeira	Consultar compras por intervalo de datas.
Relatórios > Fluxo estoque	Consultar saldo por produto e operações associadas a compras/vendas.

9. Pontos de atenção identificados no código

Os itens abaixo foram identificados por inspeção estática dos arquivos. Devem ser tratados como pendências técnicas e verificados em homologação antes de qualquer uso operacional com dados reais.

Prioridade	Achado	Efeito / correção recomendada
Crítica	Conexões e catálogos divergentes: App.config aponta para db_250405, mas há TableAdapters e formulários que usam SuperMercado ou ConnectionString1.	Padronizar uma única configuração; revisar DataSet.xsd, Settings.settings, Settings.Designer e os formulários. Rotacionar qualquer credencial que tenha sido versionada.
Crítica	Finalização da venda passa cliente 1, funcionário 1 e Pix fixos.	Usar Sessao.IdFuncionario, resolver IdCliente pelo CPF da sessão e enviar o valor selecionado no controle de pagamento. Permitir cliente nulo quando não identificado.
Alta	Login carrega FKCargo como se fosse nível de acesso.	Alterar a consulta para relacionar GA_Funcionarios com GA_Cargos e retornar GA_Cargos.Nivel.
Alta	Cadastro de produto insere na tabela Produtos, ausente no script, e não fornece todas as chaves exigidas por GA_Produtos.	Adequar INSERT para GA_Produtos, capturar fornecedor/categoria e unidade de medida e validar as FKs.
Alta	Cadastro de lote usa catálogo SuperMercado e grava apenas GA_Lote.	Apontar para db_250405 e criar GA_Estoque no mesmo fluxo/transação, ou direcionar o cadastro para a rotina de compra.
Alta	Cadastro/login de cliente tem caminhos inconsistentes: adapter associado a SuperMercado e fluxo Application.Run separado.	Centralizar conexão, preencher SessaoClientes e continuar no painel já aberto, sem iniciar outra message loop.
Média	frmAdicionar usa produtos estáticos e não persiste a confirmação.	Finalizar a integração com o banco ou identificar a tela como protótipo e removê-la do menu operacional.
Média	README lista usp_RelatorioEstoqueCritico, mas o SQL entrega vw_EstoqueCritico.	Atualizar o README ou criar a procedure e respectivos testes, mantendo um contrato único.
Média	Relatório financeiro acessa compras por TableAdapter; não agrega vendas na consulta principal observada.	Definir o escopo do relatório e incluir vendas se a intenção for demonstrar fluxo financeiro líquido/total.

Recomendações de segurança

- Armazenar senhas com hash forte e salt individual (por exemplo PBKDF2 ou BCrypt), nunca em texto puro; atualizar o login para verificar o hash.
- Remover credenciais de banco do código e do histórico Git. Utilizar configuração externa protegida e permissões mínimas no SQL Server.
- Manter consultas parametrizadas e revisar os pontos que ainda utilizam AddWithValue, com tipos e tamanhos de parâmetro explícitos.
- Implementar autorização no ponto de execução da operação, além de ocultar menus na interface.
- Registrar logs de auditoria para operações de compra, venda, cancelamento e alteração de estoque, sem gravar senhas ou dados desnecessários.

10. Manutenção, testes e solução de problemas

10.1 Checklist de testes de regressão

Área	Teste	Resultado esperado
Conexão	Abrir o app com o banco configurado.	Sem erro de conexão; catálogo db_250405 acessível.
Autenticação	Login válido e inválido.	Acesso somente com funcionário cadastrado; sessão preenchida corretamente.
Permissões	Testar cargos com diferentes níveis.	Menu administrativo refletindo GA_Cargos.Nivel.
Produto/lote	Consultar lote válido e inexistente.	Dados do produto e validade corretos; mensagens para IDs inválidos.
Estoque	Tentar quantidade acima do saldo.	Venda bloqueada; estoque não fica negativo.
Venda	Registrar venda com um e vários lotes.	Venda e itens gravados; total e baixa de estoque coerentes.
Cancelamento SQL	Cancelar venda e cancelar itens individualmente.	Itens marcados; estoque devolvido uma única vez.
Compra	Comprar itens para um e vários fornecedores.	Compra, itens, lotes e estoque consistentes; rollback se falhar.
Relatórios	Filtrar dia, mês e ano.	Período aplicado corretamente e sem datas futuras inválidas.
Integridade	Tentar CPF/CNPJ duplicados e FKs inválidas.	Banco rejeita dados inconsistentes.

10.2 Problemas frequentes

Sintoma	Causa provável	Verificação / solução
Não conecta ao LocalDB	Instância ausente, banco não criado ou cadeia de conexão incorreta.	Verificar LocalDB, App.config e existência do catálogo db_250405.
OPENJSON não reconhecido	SQL Server antigo ou nível de compatibilidade baixo.	Verificar @@VERSION e compatibility_level; ajustar o ambiente SQL.
Login sempre inválido	Funcionário não cadastrado, credenciais diferentes ou TableAdapter apontando para outro catálogo.	Consultar GA_Funcionarios e revisar a cadeia de conexão e os parâmetros.
Erro de chave estrangeira ao vender	IDs fixos inexistentes ou funcionário/cliente não cadastrados.	Corrigir parâmetros da procedure e validar os registros referenciados.
Lote sem estoque	GA_Estoque não contém linha para FKLote.	Criar estoque para o lote e corrigir o fluxo de cadastro para criar ambos em transação.
Tabela Produtos não encontrada	O cadastro envia INSERT para nome de tabela fora do esquema.	Corrigir o INSERT para a estrutura GA_Produtos.

Sintoma	Causa provável	Verificação / solução
Relatório sem dados	Banco conectado ao catálogo errado ou sem movimentações.	Conferir a conexão do TableAdapter/SqlClient e os dados de GA_Compra/GA_Venda.

10.3 Rotina de manutenção do banco

- Realizar backup antes de alterações de schema e antes de importar dados reais.
- Executar scripts de atualização em ambiente de homologação e validar procedures/triggers com transações controladas.
- Evitar atualizar manualmente GA_Estoque sem entender os gatilhos que recalculam StatusEstoque.
- Monitorar lotes próximos do mínimo por meio da view vw_EstoqueCritico.
- Manter o código gerado de DataSet sincronizado com DataSet.xsd e com o schema efetivo do banco.

11. Referência rápida de arquivos e objetos SQL

Arquivos C# relevantes

Arquivo	Responsabilidade
Program.cs	Ponto de entrada e inicialização WinForms.
forms/FrmSplash.cs	Splash screen e temporizador de abertura do login.
forms/frmLogon.cs	Autenticação do funcionário.
forms/frmPainel.cs	Painel do caixa, carrinho, estoque e chamada da procedure de venda.
forms/frmAddProdVenda.cs	Entrada de IDs de produto/lote e quantidade.
forms/frmCompra.cs	Interface e gravação de compras por fornecedor.
forms/frmCadastroFuncionario.cs	Cadastro de funcionários.
forms/frmRegistro_Cliente.cs	Cadastro e identificação de clientes.
forms/frmCadastroProduto.cs	Tela de cadastro de produto; requer adequação ao schema.
forms/frmRegistroLote.cs	Cadastro de lotes; requer correção de conexão e estoque.
forms/frmRegistroFornecedor.cs	Cadastro de fornecedores.
forms/frmRelatorioCompra.cs	Consulta de compras e seus itens.
forms/frmRelatorioEstoque.cs	Saldo consolidado por produto e histórico de movimentação.
Sessao/Sessao.cs	Dados do funcionário logado.
Sessao/SessaoClientes.cs	Dados temporários do cliente identificado.

Objetos SQL relevantes

Tipo	Nome
Tabelas	GA_Clientes; GA_Cargos; GA_Departamentos; GA_Funcionarios; GA_Fornecedor; GA_Categoria; GA_Produtos; GA_Lote; GA_Estoque; GA_Compra; GA_ItensCompra; GA_Venda; GA_ItensVenda.
Triggers	trg_UpdateTotalVenda; trg_UpdateTotalCompra; trg_BaixarEstoqueVenda; trg_AdicionarEstoqueCompra; trg_AtualizarEstoqueMinimo; trg_DevolverEstoque_ItemCancelado; trg_CancelarItensAoCancelarVenda; trg_AtualizarTotalComprasCliente; trg_AtualizarPrecoLote.
Procedures	usp_RegistrarVenda; usp_RegistrarCompra; usp_CancelarVenda; usp_CancelarItensVenda.
View	vw_EstoqueCritico.

Convenções e cuidados

- Prefixo GA_ identifica tabelas do domínio Guri Atacadista.
- FK indica chave estrangeira; Id identifica a chave primária; Data e ValorTotal representam datas e valores monetários persistidos.
- A quantidade de estoque é mantida por lote, e não diretamente por produto. Para apresentar o total por produto, somam-se os registros GA_Estoque dos lotes relacionados.
- Cancelamento de item é representado por GA_ItensVenda.Cancelado; cancelamento de venda é representado por GA_Venda.StatusVenda.

Encerramento

O sistema reúne uma base funcional para operações de mercado, com estrutura relacional, procedimentos transacionais e interface desktop. Para uma implantação confiável, as prioridades são unificar as conexões, corrigir o vínculo entre sessão e venda, alinhar os cadastros ao esquema GA_ e fortalecer o armazenamento de credenciais. Depois dessas correções, recomenda-se executar integralmente o checklist de homologação deste manual.

Anexo A - consultas de diagnóstico

As consultas abaixo são somente leitura e podem ser executadas no SSMS após selecionar o catálogo correto. Elas ajudam a verificar a estrutura e identificar lotes sem registro de estoque.

A.1 Conferir os objetos principais

```
USE db_250405;

SELECT name, type_desc
FROM sys.objects
WHERE name LIKE 'GA_%'
  OR name LIKE 'usp_%'
  OR name LIKE 'trg_%'
  OR name = 'vw_EstoqueCritico'
ORDER BY type_desc, name;
```

A.2 Localizar lotes sem estoque associado

```
SELECT l.IdLote, l.FKProduto, l.FKFornecedores, l.Quantidade, l.DataValidade
FROM GA_Lote AS l
LEFT JOIN GA_Estoque AS e ON e.FKLote = l.IdLote
WHERE e.IdEstoque IS NULL
ORDER BY l.IdLote;
```

A.3 Consultar estoque crítico

```
SELECT Produto, IdLote, DataValidade, QuantidadeAtual,
    QuantidadeMinima, FaltamParaMinimo, StatusEstoque
FROM vw_EstoqueCritico
ORDER BY FaltamParaMinimo DESC;
```

Uso seguro

Execute consultas de diagnóstico em modo somente leitura. Antes de corrigir dados, faça backup, avalie os relacionamentos e valide a alteração em homologação.